

Article

Enhancing IoT Security: Optimizing Anomaly Detection through Machine Learning

Maria Balega ^{1,2,*}, Waleed Farag ¹, Xin-Wen Wu ³ , Soundararajan Ezekiel ¹ and Zaryn Good ¹

¹ Department of Mathematical and Computer Sciences, Indiana University of Pennsylvania, Indiana, PA 15705, USA; farag@iup.edu (W.F.); sezeziel@iup.edu (S.E.); zaryngood@outlook.com (Z.G.)

² Information Networking Institute, Carnegie Mellon University, Pittsburgh, PA 15289, USA

³ Department of Computer Science, University of Mary Washington, Fredericksburg, VA 22401, USA; xwu@umw.edu

* Correspondence: mbalega@andrew.cmu.edu

Abstract: As the Internet of Things (IoT) continues to evolve, securing IoT networks and devices remains a continuing challenge. Anomaly detection is a crucial procedure in protecting the IoT. A promising way to perform anomaly detection in the IoT is through the use of machine learning (ML) algorithms. There is a lack of studies in the literature identifying optimal (with regard to both effectiveness and efficiency) anomaly detection models for the IoT. To fill the gap, this work thoroughly investigated the effectiveness and efficiency of IoT anomaly detection enabled by several representative machine learning models, namely Extreme Gradient Boosting (XGBoost), Support Vector Machines (SVMs), and Deep Convolutional Neural Networks (DCNNs). Identifying optimal anomaly detection models for IoT anomaly detection is challenging due to diverse IoT applications and dynamic IoT networking environments. It is of vital importance to evaluate ML-powered anomaly detection models using multiple datasets collected from different environments. We utilized three reputable datasets to benchmark the aforementioned machine learning methods, namely, IoT-23, NSL-KDD, and TON_IoT. Our results show that XGBoost outperformed both the SVM and DCNN, achieving accuracies of up to 99.98%. Moreover, XGBoost proved to be the most computationally efficient method; the model performed 717.75 times faster than the SVM and significantly faster than the DCNN in terms of training times. The research results have been further confirmed by using our real-world IoT data collected from an IoT testbed consisting of physical devices that we recently built.

Keywords: anomaly detection; DCNN; Internet of Things (IoT); machine learning (ML); SVM; XGBoost



Citation: Balega, M.; Farag, W.; Wu, X.-W.; Ezekiel, S.; Good, Z. Enhancing IoT Security: Optimizing Anomaly Detection through Machine Learning. *Electronics* **2024**, *13*, 2148. <https://doi.org/10.3390/electronics13112148>

Academic Editors: Abdussalam Elhanashi and Pierpaolo Dini

Received: 12 March 2024

Revised: 25 May 2024

Accepted: 27 May 2024

Published: 31 May 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

First presented by Kevin Ashton in 1999, the Internet of Things is an extension of the internet to objects or things that are traditionally not interconnected [1,2]. With the technological advances in the decades since, the number of these devices in use has grown exponentially [3]. By 2025, it is projected that there will be around 30.9 billion connected devices which is an increase from the roughly 13.8 billion devices in 2021 [4].

IoT devices are usually connected through wireless communications to various networks for which network nodes can transmit data and interact with one another or with centralized devices. As the Internet of Things continues to evolve, it poses risks and the need for greater security measures. Securing IoT networks and devices is a constant challenge as the development of the IoT is moving faster than the creation of defenses for the devices and users themselves [3,5].

The deployment of IoT applications makes protection more challenging with the increased attack surface as well as the vulnerable and resource-constrained end devices. IoT applications pose a greater attack surface due to the extensive connectivity as well as the deployment of end devices that are not protected or minimally protected by the security

protocols commonly applied to traditional computer networks. Additionally, many IoT networks and end devices do not have sufficient computational capabilities to integrate advanced firewalls, antivirus software, and authentication processes. Furthermore, in IoT applications connected with important systems or critical infrastructure, real-time security monitoring and instant risk responses are highly desirable. With these vulnerabilities, constraints, and risk response requirements, securing IoT applications remains a significant challenge [6,7].

Anomaly detection is a crucial procedure in protecting networked systems as it allows for the identification of unusual or abnormal behavior within a network of connected devices. This is particularly important in the context of the IoT, where the sheer volume of data generated by vulnerable devices can make it difficult to identify and respond to potential security threats [8,9].

1.1. Contributions

Machine learning (ML) methods show promise for anomaly detection in IoT systems, as observed in the related works detailed in the next section. Although existing works explored the applications of various classical ML models to IoT anomaly detection, there is a lack of investigations regarding the efficiency of ML-powered anomaly detection. Also, some of the most promising ML models, such as Extreme Gradient Boosting (XGBoost), were not thoroughly investigated with regard to the detection accuracy and efficiency. There is a great gap in this area with regard to identifying optimal (with regard to both effectiveness and efficiency) anomaly detection models for the Internet of Things. To fill the gap, this work thoroughly investigated the effectiveness and efficiency of IoT anomaly detection enabled by several representative machine learning models, namely XGBoost, Support Vector Machines (SVMs), and Deep Convolutional Neural Networks (DCNNs).

Previous studies show that SVMs, DCNNs, and several other ML models are effective in classifying data and detecting anomalies [6,7]. However, these models were not thoroughly studied regarding efficiency. Identifying the optimal anomaly detection models for the IoT is indeed challenging due to the diverse IoT applications and dynamic IoT networking environments. Even if only considering a specific type of IoT application, such as IoT-facilitated smart buildings or intelligent transportations systems, the IoT systems working in dynamic environments produce very different data. Therefore, it is of vital importance to evaluate ML-powered anomaly detection models using multiple datasets collected from different environments. We utilized three reputable datasets to benchmark the machine learning methods, namely, IoT-23, NSL-KDD, and TON_IoT [10–12]. IoT-23 is a dataset that contains 23 different captures of network-based attacks on IoT devices, making it a useful resource for evaluating the performance of anomaly detection algorithms. The NSL-KDD dataset was specifically designed for use in network security research, and it contains a wide range of network-based attacks. TON_IoT is a dataset that is designed to reflect the traffic patterns of IoT devices, making it a valuable resource for understanding the behavior of IoT networks. These datasets have proven to be a great representation of real-world IoT systems and attacks [13].

In this study, each machine learning algorithm was assessed based on accuracy, precision, recall, and F1 score. Our experimental results show that the XGBoost algorithm outperformed both the SVM and DCNN algorithms, achieving accuracies up to 99.98%. We further studied the efficiency of anomaly detection models powered by these learning algorithms. XGBoost proved to be the most efficient method with respect to execution time. While DCNNs and SVMs are more computationally expensive, XGBoost trains data much more quickly. As XGBoost is capable of handling large amounts of data effectively and efficiently, it proves to be the best anomaly detection model when compared against SVMs and DCNNs.

Recently, we have built a testbed with physical IoT devices, including cameras, an Amazon echo, smart plugs, and other IoT sensors and collected data from them. We have tested the machine learning models using our real-world IoT data, and our results

confirm and support the research results obtained through using the IoT-23, NSL-KDD, and TON_IoT datasets.

Our evaluation of ML-powered anomaly detection models using real-world data proves that XGBoost can be used to detect anomalies efficiently and accurately in real-world IoT applications. The results of this research are expected to be helpful to individual users and organizations to identify and implement the most effective and efficient anomaly detection systems to secure their IoT applications. Our study offers invaluable insights directly applicable to protecting IoT applications, thereby enhancing the overall security and resilience of these systems and enabling users and organizations to take proactive measures in safeguarding them.

In summary, the contributions of this paper include (1) evaluation of machine learning-powered anomaly detection models for the Internet of Things regarding both effectiveness and efficiency; (2) investigation of XGBoost in IoT anomaly detection applications and proof that it is an effective and efficient model; (3) utilization of distinct datasets in an evaluation of ML-powered IoT anomaly detection models and verification of the models using real-world data collected from an IoT testbed that we recently built; and (4) identification of an optimal anomaly detection model, enabling users and organizations to take proactive measures in safeguarding their IoT applications.

1.2. Paper Layout

The rest of the paper is organized as follows. In Section 2, works related to this study will be presented. In Section 3, the anomaly detection models and datasets used in this study will be described as well as the preprocessing and training of each dataset. In Section 4, the evaluation metrics used for assessing each model will be presented. In Section 5, the results from each of the machine learning models will be compared. Efficiency results, verification of the models using real-world data, and limitations will be discussed in Section 6, and concluding remarks and future work will be presented in Section 7.

2. Related Works

Datasets from real-world IoT applications are not readily available for research purposes, primarily due to privacy concerns. The lack of availability of labeled IoT datasets presents a difficulty in the development of anomaly or intrusion detection systems [14]. Because of this gap, IoT datasets have been produced recently in the research community, including those used in our study.

An attempt to apply machine learning for securing the IoT was reported in [15]. Due to the large amount of network traffic and data collected through various IoT devices, researchers realized that highly effective algorithms are desirable for accurate classification and regression when using machine learning for IoT security [16]. Supervised and unsupervised machine learning methods were further investigated for enhancing IoT security [17].

Several popular machine learning methods and deep learning models, namely, Support Vector Machines (SVMs), k-Nearest Neighbor (kNN), Naïve Bayes (NB), Random Forest (RF), Classification and Regression Trees (CARTs), Logistics Regression (LR), and Linear Discriminant Analysis (LDA) were evaluated regarding the performance of intrusion detection using the TON_IoT dataset [14]. As TON_IoT is made up of smaller datasets, this study compared the performance of the detection models on each dataset as well as a combination of them. The results show that for binary clarification and multiclassification on the combined datasets, CARTs performed best. Specifically, for binary classification, accuracies up to 88% were achieved by this detection model; for multiclassification, accuracies up to 77% were achieved. Random Forest was the second best, achieving accuracies up to 85% for binary classification and 71% for multiclassification. Overall, this study found that CARTs and Random Forest achieved the highest score in all evaluation metrics with separate datasets as well as when the datasets were combined [14].

In 2021, machine learning algorithms such as SVM, Gradient Boosting techniques, Isolation Forest, and Deep Learning Networks were assessed in terms of their capabilities for detecting intrusion using the IoT-23 dataset [18]. In a recent paper [19], the authors provided a review of existing works on developing anomaly detection for the IoT using the KNN, SVM, Bayesian Network, and Neural Network machine learning algorithms.

Recently, interesting results on IoT intrusion detection were published [8,9]. In these papers, the authors examined NetFlow features and assessed their suitability for classifying network traffic. They utilized multiple datasets, containing IoT traffic data represented in a standard set of 43 NetFlow-based features. Reducing the number of features from 43 to 7 can enhance the prediction time and, consequently, the performance in the real world. With the reduced number of features, their network intrusion detection system (based on an ML model boosted by a modified version of the Arithmetic Optimization Algorithm) achieved up to 99% and 98% accuracy for binary and multi-classification, respectively.

However, the efficiency of anomaly detection of different detection models was not thoroughly investigated in these works. In [7], published in 2022, the authors studied the effectiveness as well as efficiency of the following ML models for IoT anomaly detection: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting Machine, and Naïve Bayes. However, like the abovementioned papers, they only evaluated these ML models using a single dataset, that is, a Kaggle dataset. Also, this paper did not evaluate the well-known SVM and DCNN and only studied binary classification anomaly detection.

In our previous work, ML anomaly detection models for IoT were evaluated using the IoT-23 dataset [20], and the models were evaluated using the NSL-KDD dataset in another paper [21]. The present paper will extend these results to include more datasets. A comparison of previous publications and this present paper can be found in Table 1.

Table 1. Comparison of prior publications vs. this paper.

Publications vs. This Paper	Dataset Features Selected (to Increase Detection Accuracy)?	Identified the Optimal Detection Model Regarding Accuracy?	Multiple Datasets Used?	Detection Efficiency Thoroughly Studied?
[7]	N/A *	Compared ML models regarding accuracy and identified the most accurate ML model	No	Compared ML models regarding accuracy and identified the most accurate ML model
[8]	Yes	Studied only one detection model	Yes	N/A
[9]	Yes	Studied only one detection model	Yes	N/A
[15]	No	Yes	No	No
[19]	No	Yes	No	No
[20]	No	Yes	No	No
[21]	No	Yes	No	No
This paper	Yes	Compared ML models regarding accuracy and identified the most accurate ML model	Used multiple datasets to evaluate the ML models' performance in different environments	Compared ML models regarding accuracy and identified the most accurate ML model

* The datasets used in [7] contain a small number of features, therefore feature selection was unnecessary.

With diverse IoT applications and dynamic IoT networking environments, IoT systems may produce very different data. Therefore, it is of vital importance to evaluate ML-powered anomaly detection models using more than one dataset collected from distinct environments. This motivates us to investigate the most accurate anomaly detection models

that are effective on different datasets, and we strive to identify the most efficient models that suit IoT applications with resource-constrained devices.

3. Materials and Methods

3.1. Learning Models for Anomaly Detection

Machine learning algorithms can be broken into three primary parts: a decision process that makes a prediction or classification based on some input data, an error function to evaluate the predictions and adjust for accuracy, and a model optimization process that adds or adjusts weights to various factors to reduce discrepancies between the model's estimates and the known examples. Using the input data, whether labeled or unlabeled, a learning algorithm creates an estimate of patterns discovered in the data. The error function then evaluates the prediction of the model, comparing the known examples to assess the model's accuracy. If the model can better fit data in the training set, then the weights are adjusted to optimize the model. This process is repeated until a threshold accuracy is found [22].

In this study, supervised machine learning models were used which apply the use of labeled datasets to classify data. The learning models chosen include XGBoost, SVMs, and DCNNs. These models have proven to be effective as they have been applied in many machine learning challenges. Particularly, XGBoost has been widely used by data scientists and it has shown promising results [23].

3.1.1. Extreme Gradient Boosting

Extreme Gradient Boosting or XGBoost is an implementation of Gradient Boosting which is used for regression and classification problems. Gradient Boosting creates a prediction model through an ensemble of weak prediction models. It generalizes them using the differentiable loss function [24]. XGBoost differs from Gradient Boosting in that it uses a more regularized model to help control overfitting. It considers the complexity of the model, adding terms to limit the growth of the tree which aids in better performance [25].

XGBoost minimizes an objective function (Equation (1)) which combines the loss function, L , and a penalty or regularization term, Ω , for the complexity and overfitting. This is an iterative process where new trees are added to predict the errors of prior trees. These are all combined to make the final prediction. The loss function (Equation (2)) used is the logistic loss, where y_i is the labels.

$$\text{obj}(\theta) = L(\theta) + \Omega(\theta) \quad (1)$$

$$L(\theta) = \sum_i [y_i \ln(1 + e^{-y_i}) + (1 - y_i) \ln(1 + e^{y_i})] \quad (2)$$

XGBoost uses general parameters, booster parameters, and learning task parameters. General parameters relate to the booster being used. This is typically a tree or linear model. Booster parameters are chosen based on the specific type of booster being used. Significant booster parameters are η and max depth. η is the learning rate which helps to prevent overfitting the data. η ranges from zero to one. Max depth refers to the maximum depth of the tree. Increasing this value creates a more complex model. Max depth ranges from zero to infinity. Learning task parameters are based on the learning scenario. With multiclassification problems, the learning task parameter SoftMax is used [26].

3.1.2. Support Vector Machines

Support Vector Machines focus on mapping inputs non-linearly to a high-dimension feature space, Z , where a linear decision surface is constructed. The goal is to find an optimal hyperplane separating the data as generally as possible. In 1965, Vapnik solved an optimal hyperplane for separable classes [27]. The small amount of training data or support vectors determines the margin of the largest separation. The training data are separated into different groups with small training vectors in each proportion while solving

the quadratic objective function. This will determine whether either the portion cannot be separated, or an optimal hyperplane has been found.

Typically, there is no exact single line that perfectly divides real data, and a curved boundary is used. For non-linear inputs, the “kernel trick” can be used. The kernel is defined by $K(x, x')$, where x and x' are the inputs and K is the kernel used [28]. There are a variety of common kernels to choose from. The linear kernel (Equation (3)) is the fastest but least accurate for multiple classes. The polynomial kernel (Equation (4)) is used for non-linear models, where d is the degree of the polynomial. Most often used is the radial basis function (Equation (5)), as it overcomes the space complexity issues of SVMs. Sigmoid or Multi-Layer Perceptron (Equation (6)) uses a single layer, where ρ is the slope and g is the intercept. This kernel is preferred in Neural Networks.

$$K(x, x') = x \langle x, x' \rangle \quad (3)$$

$$K(x, x') = (\langle x, x' \rangle + 1)^d \quad (4)$$

$$K(x, x') = \exp \frac{-||x - x'||^2}{2\sigma^2} \quad (5)$$

$$K(x, x') = \tanh(\rho \langle x, x' \rangle + g) \quad (6)$$

When creating a classifier, tuning parameters C and Γ are used. C controls the tradeoff between a smooth decision boundary and correctly classifying training points. Γ defines how far the influence of a single training example reaches. When set to a lower value, this means every point has a far reach. When set to a higher value, this means each point has a close reach. Γ ranges in value from zero to one [29].

3.1.3. Deep Convolutional Neural Networks

Deep Convolutional Neural Networks or DCNNs use a three-dimensional neural pattern to identify patterns in image and video data. DCNNs are comprised of various types of layers that each perform specific tasks. The first type of layer is the convolutional layer which passes a filter over the image multiple times, performing matrix multiplication with weights and inputs. These values are added together to obtain a value for each filter position.

Another type of layer is the non-linear activation layer which removes negative numbers from the convolutional layer then maps and replaces them with zeros. The pooling layer is used to shrink the image by removing noncritical information. This can be achieved using max pooling or average pooling. With max pooling, only the maximum value in a group of pixels is preserved. With average pooling, only the average value in a group of pixels is preserved.

The fully connected layer is the final type of layer. The fully connected layer takes an input vector of the filtered images and reduced pixels that have been flattened and applies a SoftMax function. This layer is what classifies the image.

The DCNN architectures used in this study were AlexNet, GoogleNet, ResNet-50, VGG16, and VGG19 [30]. The number of layers for each of these networks is presented in Table 2.

Table 2. DCNN network architectures.

Network	Depth/Layers	Parameters	Image Size
AlexNet	8	61	227 × 227
GoogleNet	22	4	224 × 224
ResNet-50	50	23	224 × 224
VGG16	16	138	224 × 224
VGG19	19	144	224 × 224

3.2. Datasets

In this study, three datasets containing data from IoT devices and networks were adopted to perform anomaly detection using machine learning algorithms. The first dataset used was the IoT-23 dataset captured by the Stratosphere Laboratory [6]. This dataset was created for researchers to develop and apply machine learning algorithms. The second dataset used was the NSL-KDD dataset, which is an improved version of the KDD '99 dataset. The NSL-KDD dataset is suggested to solve some inherent problems in the KDD '99 dataset. While this new version still suffers from some problems, it is still an effective benchmark dataset that can be used for research [11]. Lastly, the TON_IoT dataset was used. This dataset consists of new generations of IoT and Industrial IoT data for evaluating the fidelity and efficiency of various cybersecurity applications such as artificial intelligence and machine learning [12].

3.2.1. IoT-23 Dataset

The IoT-23 dataset consists of network traffic from IoT devices. It contains 20 malicious captures and three benign captures. The malware captures are from infected IoT devices, and the benign captures are from real IoT devices. For the malicious captures, a specific malware sample was executed in a Raspberry Pi. Data for benign captures were captured through IoT devices, including a Philips HUE smart LED lamp, an Amazon Echo, and a Somfy smart door lock. These three IoT devices are real hardware and were not simulated [10].

Each capture in the dataset has 23 features. Of these 23 features, the use of 13 were focused on in this study. For a complete list of the columns and their descriptions, see [31]. This entire dataset is roughly 90.5% malicious traffic, containing a total of 325,307,990 entries [31]. Multiple types of malicious labels were created by the Stratosphere Lab. The most common labels include PartOfAHorizontalPortScan, Okiru, and DDoS and the least common include C&C-Mirai, PartOfAHorizontalPortScan-Attack, and C&C-HeartBeat-FileDownload [10].

3.2.2. NSL-KDD Dataset

The NSL-KDD dataset was built based on data captured in the DARPA '98 IDS evaluation program and is an improved version of the KDD CUP '99 dataset [32]. DARPA '98 consists of roughly four gigabytes of compressed binary tcpdump data captured from seven weeks of network traffic. The KDD CUP '99 training data consists of around 4,900,000 entries containing 41 features and are labeled as normal or attack [33].

While the KDD CUP '99 dataset has been used for many years, it encounters numerous problems. For example, synthetic data were used for the background and attack data when creating the dataset. The data also do not represent real networks as the workload of data does not seem similar to traffic that occurs in real networks. Further, data are only categorized as normal or attack rather than by the type of specific attack [33]. The KDD CUP '99 dataset also contains a large number of redundant records, as about 78% and 75% are duplicated in the train and test sets, respectively [34]. To address these issues, the NSL-KDD dataset was created.

3.2.3. TON_IoT Dataset

The TON_IoT dataset was created in response to testing the efficiency and preciseness of cybersecurity applications, including the use of machine learning algorithms. This dataset is used for validating and testing applications such as intrusion detection systems, threat intelligence, malware detection, fraud detection, privacy preservation, digital forensics, adversarial machine learning, and threat hunting [12].

The TON_IoT dataset consists of several datasets where data were collected from Telemetry datasets of IoT services, operating systems such as Windows and Linux, and network traffic. The IoT data were collected from more than ten IoT sensors. The Linux data were collected by running a tracing tool on Ubuntu 14 and 18 systems. Finally, the Windows data were captured on Windows 7 and 10 systems. These datasets were preprocessed as

csv files with a label indicating normal or attack [35]. The types of attacks present include Backdoor, DDoS, Injection, Normal, Password, Ransomware, Scanning, and XSS.

3.3. Data Preprocessing and Model Training

3.3.1. Preprocessing the IoT-23 Dataset

In this study, we focused on the use of the conn.log.labeled files from the IoT-23 dataset. Each conn.log.labeled file was converted into a csv file and then the twenty-three csv files were combined into one large csv file. The number of benign and malicious items was calculated to create balanced data. As the IoT-23 dataset contains mostly malicious data, a random sample was taken from the malicious data that equates to the amount of benign data. The total number of entries including benign and malicious entries after random sampling was 61,721,287.

Next, the empty values containing a dash were replaced with 'NaN' so that the machine learning algorithms could handle them. Each column was assessed to determine whether it would be useful for predicting anomalies. Columns containing over 70% NaN values were dropped along with columns containing unnecessary and repetitive information. Then, categorical data were label encoded and the detailed label column was encoded to create four total classifications, as shown in Table 3.

Table 3. IoT-23 classifications.

Label Encoding	Type of Capture
0	Benign
1	DDoS
2	Okiru
3	PartOfAHorizontalPortScan

These four specific captures were the most accounted for in our sample. As there were other detailed labels in our sample, these were dropped because there were very few of them. We also included IP addresses where the attack happened and those of the device on which the capture happened. Due to the unique response IPs, we chose to include these features in the data.

As SVMs cannot process NaN values like XGBoost, all rows of data containing any NaN value were dropped, leaving 13,183,092 entries for training. Additionally, float values in the duration column exceeded the values that SVM could process, so it was dropped as well. After preprocessing, 12 features were assessed with 13,183,092 entries of data which were used to train the XGBoost and DCNN models. However, due to technical limitations, samples of 12%, 16%, and 20% were taken for the SVM model. As for the distribution of the dataset, 32.1% of the data were benign, 44.6% were PartOfAHorizontalPortScan, 10.9% were Okiru, and 12.4% were DDoS.

3.3.2. Preprocessing the NSL-KDD Dataset

For the NSL-KDD dataset, the data were already in csv file format. The data were also already split into training and testing files. Empty values that contained a dash were replaced with 'NaN'. Then, each column was assessed to determine whether it would be useful for predicting anomalies. Numerous columns were dropped as they statistically did not contribute to the training data or there was an overwhelming number of NaN values. Once the unnecessary columns were dropped, categorical data were label encoded, with the label column consisting of 23 classifications of which 22 were malicious and one was normal. There was no inclusion of IP addresses in the preprocessed data to avoid bias. After preprocessing the NSL-KDD dataset, 27 features were assessed with 125,973 entries of data containing no NaN values. As for the distribution of the NSL-KDD dataset, 53% of the data were normal, whereas the other 47% consisted of many types of malicious classifications. A notable classification was malicious—neptune which comprised 33% of the malicious data.

3.3.3. Preprocessing the TON_IoT Dataset

Each of the datasets that make up the entirety of TON_IoT were preprocessed separately, taking into consideration their unique features. A variety of one-hot, label, and cyclical encoding were used as well as scaling techniques. Specifically, min-max scaling was used, which is scaling within a range typically from zero to one. Zero-mean unit-variance scaling was also applied. For each dataset, the time stamp feature was cyclically encoded, and the date and time features were removed as they were not relevant. Each type of attack was label encoded. Each dataset was also checked for duplicate entries. If present, they were removed. With the TON_IoT data, the NaN values were treated differently than the other datasets. The mean value for the column was used to fulfill NaN values rather than drop them. There was no inclusion of IP addresses in the preprocessed data to avoid bias. Once each dataset was preprocessed, they were combined into a single csv file to run through the algorithms. After preprocessing, 24 features were accounted for with 303,855 entries of data. As for the distribution of the TON_IoT dataset, 52% of the data were benign, while the other 48% consisted of malicious data.

3.3.4. Training XGBoost

Using XGBoost, the label of the IoT data establishing whether the network traffic is benign or malicious is set as the control variable so the model can perform classification. The model is then trained using the preprocessed data and a variety of hyperparameters. Since this study focuses on classification, the parameters that were adjusted for the tree booster were η and max depth. η performed best in a range from 0.3 to 0.35. As for max depth, the default value of six was used in our study. SoftMax was also used along with the num class parameter. Additionally, the multiclass classification error rate (Equation (7)) was also calculated.

$$\text{merror} = \frac{\text{wrong cases}}{\text{all cases}} \quad (7)$$

The IoT-23 data, NSL-KDD data, and TON_IoT data were split into 80% training and 20% testing, 75% training and 25% testing, and 65% training and 35% testing to see how well the XGBoost model could classify anomalies. When training the data, the number of epochs was set to 100.

3.3.5. Training SVMs

Multiple kernels were implemented for the NSL-KDD data while the RBF kernel was mainly focused on using the IoT-23 data and TON_IoT data. C was set to a value of 10, and gamma was set to 0.01.

The IoT-23 data, NSL-KDD data, and TON_IoT data were split into 80% training and 20% testing, 75% training and 25% testing, and 65% training and 35% testing to see how well the SVM model could classify anomalies.

3.3.6. Training DCNN

With XGBoost and SVM, there is a set number of iterations the model conducts to train data, regardless of whether there is an improvement in the performance. DCNNs, however, can stop training before MaxEpochs is reached. Our DCNN is formatted to halt once there is no further improvement in regard to the accuracy. Once the model decides it is optimized, training is complete.

For DCNN parameters, the MiniBatchSize was set to 10 and the MaxEpochs was 5. The InitialLearningRate was set to 0.0001. Our ValidationFrequency was set to 5 and ValidationPatience was set to 10. The IoT-23 data, NSL-KDD data, and TON_IoT data were split into 80% training and 20% testing, 75% training and 25% testing, and 65% training and 35% testing to see how well the XGBoost model could classify anomalies. Additionally, a 70% training and 30% testing set was included for the DCNN.

4. Evaluation Metrics

Various metrics were calculated when running XGBoost, the SVM, and the DCNN. As Python was used for the SVM and XGBoost algorithms, the metrics were calculated using scikit-learn functions. MATLAB was used to run the DCNN, and these metrics were calculated based on confusion matrices.

First, the balanced accuracy (Equation (8)) was calculated as it accounts for imbalanced data. Using balanced accuracy is better in this case since it accounts for both the positive and negative outcome classes. The F1 score (Equation (9)) was another metric taken into consideration. F1 score is the mean of precision (Equation (10)) and recall (Equation (11)). It is a popular metric as it is a combination of two other important metrics. These metrics can be calculated as follows [36]:

$$\text{Balanced Accuracy} = \frac{TPR + TNR}{2} \quad (8)$$

$$\text{F1 Score} = 2 \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})} \quad (9)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (10)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (11)$$

$$\text{TPR} = \frac{TP}{TN + FP} \quad (12)$$

$$\text{TNR} = \frac{TN}{TN + FP} \quad (13)$$

where *TP* is True Positive, *FP* is False Positive, *TN* is True Negative, *FN* is False Negative, *TPR* is the True Positive Rate (Equation (12)), and *TNR* is the True Negative Rate (Equation (13)).

5. Results

5.1. Detection Performance of XGBoost

Our experimental results proved that XGBoost can predict anomalies very effectively. XGBoost achieved balanced accuracies of up to 99.98% with three different training and testing splits using the IoT-23 data, see Figure 1. With the NSL-KDD dataset, balanced accuracies of up to 80.3% were achieved using three training and testing splits, see Figure 2. As for the TON_IoT data, balanced accuracies of up to 99.90% were achieved using 75% of the data for training and 25% for testing, see Figure 3. In regard to other metrics, precision for IoT-23 was up to 99.99%, up to 89.3% using the NSL-KDD data, and up to 99.78% using the TON_IoT data. Recall was up to 99.98% for IoT-23, up to 80.3% for NSL-KDD, and up to 99.83% for TON_IoT. Further, the F1 score was up to 99.99% for IoT-23, up to 81% for NSL-KDD, and up to 99.84% for TON_IoT.

5.2. Detection Performance of SVM

SVM achieved high balanced accuracies of up to 96.71% using the IoT-23 dataset, as shown in Figure 4. Balanced accuracies of up to 77.07% were achieved with the 75/25 split using the NSL-KDD data, see Figure 5. As different kernels were assessed, the linear kernel achieved the best results using this dataset. As for the TON_IoT data, accuracies of up to 96.72% were achieved using an 80/20 split, see Figure 6. These results were gathered with the application of the RBF kernel.

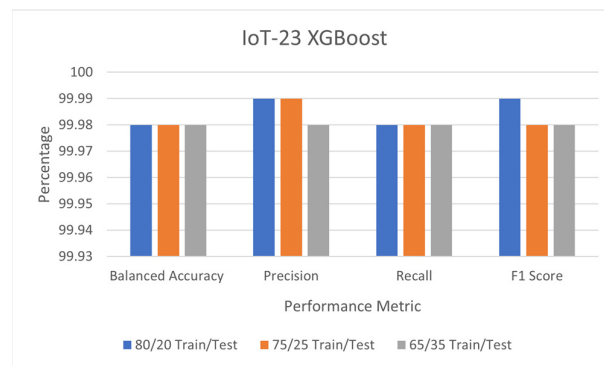


Figure 1. XGBoost performance results using the IoT-23 dataset.

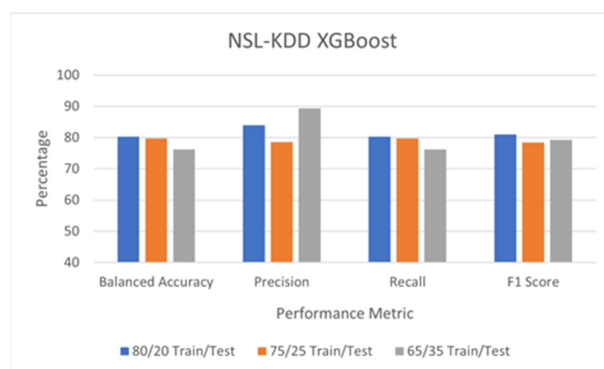


Figure 2. XGBoost performance results using the NSL-KDD dataset.

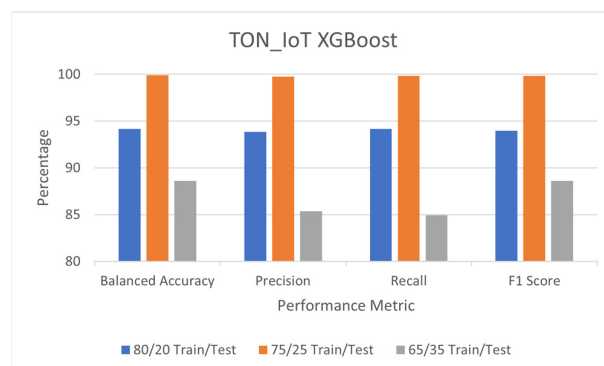


Figure 3. XGBoost performance results using the TON_IoT dataset.

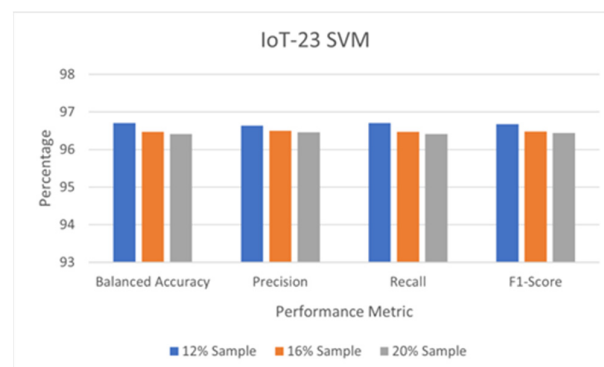


Figure 4. SVM performance results using the IoT-23 dataset.

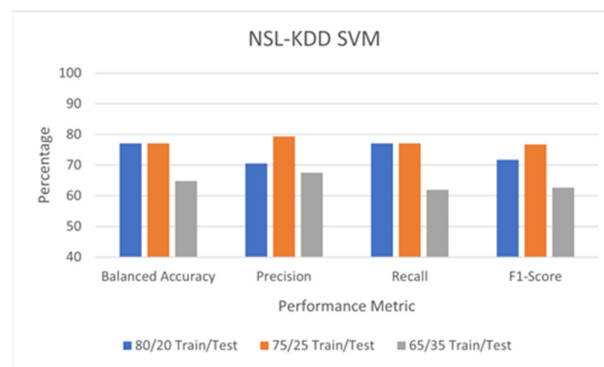


Figure 5. SVM performance results using the NSL-KDD dataset.

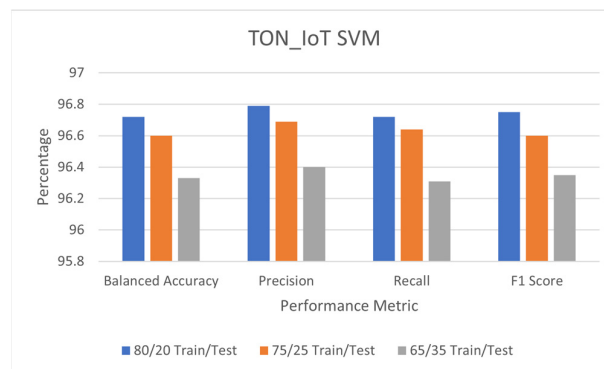


Figure 6. SVM performance results using the TON_IoT dataset.

5.3. Detection Performance of the DCNN

The IoT-23 dataset performed very well across various DCNN architectures. The highest accuracy achieved was 99.90% using the VGG16 architecture. As for the NSL-KDD dataset, it performed the best using the ResNet-50 architecture, achieving accuracies of up to 96.19%. Using the TON_IoT dataset, the ResNet-50 architecture also performed the best, with an accuracy of 92.98%. While the accuracies remained above 85% for each architecture when applying the IoT-23 and NSL-KDD datasets, there was a wider range in accuracy when using the TON_IoT dataset. The best accuracy for each architecture appears in bold in Tables 4–6.

5.4. Comparison of Detection Performance

When comparing the detection of anomalies with the models assessed, XGBoost achieved the best performance, with the highest accuracy of 99.98%. The DCNN performed the second best, achieving the highest accuracy of 99.90%. SVM was the third best algorithm, achieving accuracies up to 96.72%. Although the SVM and DCNN both achieved good accuracies, XGBoost performed well above these two models in our evaluation. In particular, the highest accuracies were found when using the IoT-23 dataset. These results can be seen in Figures 7–9.

Table 4. DCNN performance results using the IoT-23 dataset.

DCNN	Accuracy	Precision	Recall	F1 Score
AlexNet				
80/20	94.59	88.10	72.99	79.84
75/25	95.94	96.64	76.19	85.21
70/30	97.13 *	96.43	92.01	88.64
65/35	95.87	97.06	75.99	85.24
GoogleNet				
80/20	98.37	98.61	90.49	94.38
75/25	97.47	98.07	81.77	89.18
70/30	99.16 *	96.64	97.41	97.03
65/35	85.96	42.95	49.86	46.15
ResNet-50				
80/20	99.85 *	99.28	99.48	99.38
75/25	99.81	99.45	99.11	99.28
70/30	99.79	99.63	98.98	99.30
65/35	99.48	98.60	99.36	98.98
VGG16				
80/20	99.83	99.54	99.35	99.44
75/25	99.64	99.39	98.21	98.80
70/30	99.42	96.65	99.02	97.82
65/35	99.90 *	99.86	99.31	99.59
VGG19				
80/20	99.63	99.67	97.42	98.53
75/25	98.22	99.00	91.44	95.07
70/30	99.69 *	99.61	97.93	98.76
65/35	99.21	99.45	91.17	96.74

* These were the highest accuracies achieved by each architecture using the IoT-23 dataset.

Table 5. DCNN performance results using the NSL-KDD dataset.

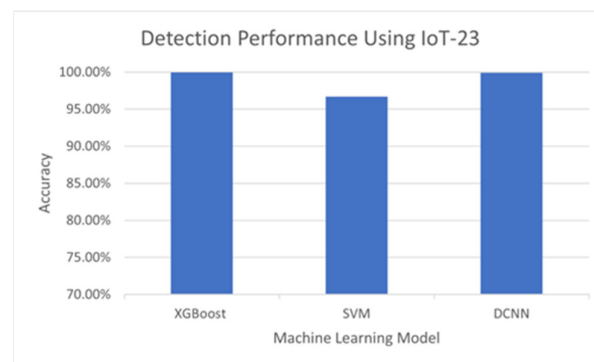
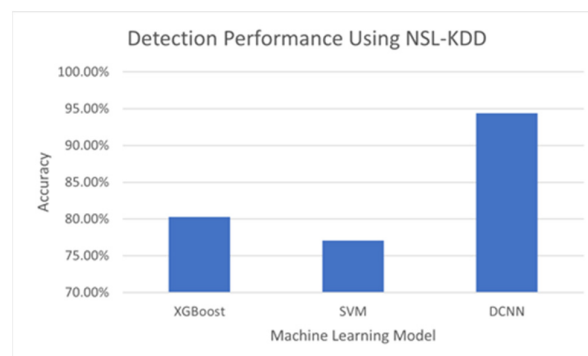
DCNN	Accuracy	Precision	Recall	F1 Score
AlexNet				
80/20	90.60	90.53	90.63	90.58
75/25	90.53	90.50	90.70	90.60
70/30	90.79 *	91.23	90.45	90.84
65/35	90.59	90.82	90.34	90.58
GoogleNet				
80/20	93.49	93.42	93.57	93.49
75/25	92.81	92.84	92.71	92.77
70/30	92.15	92.16	92.36	92.26
65/35	93.75 *	93.69	93.80	93.74
ResNet-50				
80/20	96.19 *	96.15	96.21	96.18
75/25	94.86	94.84	94.83	94.84
70/30	94.71	94.86	94.55	94.70
65/35	94.66	94.60	94.77	94.69
VGG16				
80/20	95.20 *	95.41	95.01	95.21
75/25	94.33	94.38	94.23	94.30
70/30	93.20	93.23	93.44	93.33
65/35	94.78	94.96	94.60	94.78
VGG19				
80/20	92.97	93.02	93.22	93.11
75/25	93.95	94.03	93.83	93.93
70/30	95.02	94.96	95.07	95.02
65/35	95.23 *	95.21	95.19	95.20

* These were the highest accuracies achieved by each architecture using the NSL-KDD dataset.

Table 6. DCNN performance results using the TON_IoT dataset.

DCNN	Accuracy	Precision	Recall	F1 Score
AlexNet				
80/20	64.08	69.66	65.07	67.29
75/25	67.02	75.48	68.12	71.61
70/30	92.29 *	92.71	92.11	92.41
65/35	89.14	91.30	88.70	89.98
GoogleNet				
80/20	54.80	55.98	55.42	55.70
75/25	61.03	63.15	61.71	62.42
70/30	87.83 *	89.26	87.46	88.35
65/35	81.04	81.01	81.01	81.01
ResNet-50				
80/20	92.98 *	92.96	93.01	92.98
75/25	90.82	91.38	90.6	90.99
70/30	88.93	91.13	88.48	89.78
65/35	89.03	91.22	88.57	89.88
VGG16				
80/20	84.98 *	86.24	84.60	85.41
75/25	63.39	68.40	64.35	66.32
70/30	70.98	80.84	72.08	76.21
65/35	74.81	78.53	72.08	76.21
VGG19				
80/20	69.88	73.52	70.62	71.04
75/25	71.79	72.71	72.14	72.43
70/30	86.76	88.41	86.34	87.37
65/35	88.85 *	91.12	88.39	89.74

* These were the highest accuracies achieved by each architecture using the TON_IoT dataset.

**Figure 7.** Machine learning model comparison using the IoT-23 dataset.**Figure 8.** Machine learning model comparison using the NSL-KDD dataset.

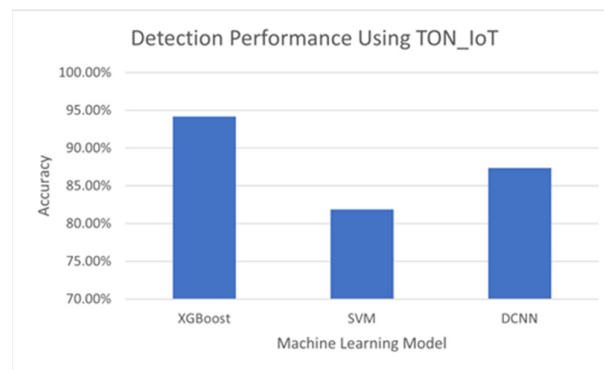


Figure 9. Machine learning model comparison using the TON_IoT dataset.

6. Efficiency, Real-World Data, and Limitation Discussion

6.1. Efficiency

In terms of CPU time, XGBoost proved to be the most efficient algorithm. By changing the parameters of the machine learning models, the models might obtain slightly better results; however, training takes an extremely long amount of time, particularly for the DCNN and SVM.

Training the DCNN is much more time-intensive than training both the SVM and XGBoost. One reason for this is because the training inputs are image data rather than raw numerical data. The larger size of the images means that it is more computationally expensive to use than relatively small-sized numerical data. Additionally, the DCNN uses a multilayer approach, which requires more time for each input. Some images will also be used in the training process multiple times. However, once the training is complete, testing the trained network is much quicker and more efficient, taking a small fraction of the time it takes to originally train it.

Since the TON_IoT and NSL-KDD preprocessed data were consistently used across all three models, the averages of the 80/20 training and testing split times from these datasets were calculated to compare each model's efficiency, as shown in Table 7. Each of these models were run on the same machine under equivalent conditions. XGBoost took an average of 0.7748 s for training and only 0.0032 s for testing. The SVM took an average of 556.11 s for training and 79.06 s for testing. As for the average across all DCNN architectures, the training time was about 9916.9 s and the testing time was 342.49 s. Therefore, XGBoost proves to be 717.75 times faster than the SVM and 12,799.3 times faster than the DCNN in regard to training times.

Table 7. ML model efficiency results.

	XGBoost	SVM	DCNN
Training (s)	0.7748	556.11	9916.9
Testing (s)	0.0032	79.06	342.49

6.2. Verification with Real-World Data

As part of our ongoing project, we recently built an IoT testbed as previously mentioned. Using our real data collected from the IoT testbed, we applied the machine learning models used in this study. Balanced data was used, where there was a total of 320,000 captures and 12 features. Various training and testing splits were assessed including 80/20, 75/25, 70/30, and 65/35. XGBoost proved to perform the best overall, achieving accuracies up to 99.97%, whereas the SVM achieved accuracies of up to 95.55%. Using the ResNet-50 architecture, the DCNN achieved accuracies of up to 98.46%.

As far as execution times are concerned, training the data using XGBoost took 1.71 s on average, while testing took about 0.0058 s. The SVM took about 981.02 s for training and 220.74 s for testing. Lastly, the DCNN took an average of 23,598.96 s for training

and 342.21 s for testing. The results using our testbed data strongly support our results previously discussed in Section 5.

6.3. Limitation Discussion

In this paper, we studied IoT anomaly detection powered by ML models using multiple publicly available datasets (as well as real-world data collected from the smart building testbed that we recently built) from distinct IoT systems to evaluate the effectiveness of the models regarding dynamic IoT environments. Not only did we examine the accuracy of anomaly detection, but we also thoroughly assessed the efficiency that is highly desired for resource constrained IoT applications (but not systematically addressed by any prior works). We focused on three representative ML models, namely, Extreme Gradient Boosting, Support Vector Machines, and Deep Convolutional Neural Networks, to identify the optimal model with regard to both the detection accuracy and efficiency based on our selected datasets. However, as discussed in previous sections, IoT systems are applied to various dynamic environments that produce very different data. We do not expect any single detection model (such as XGBoost that we identified in this research as the optimal model for the datasets that we used) will work most effectively for all IoT systems with different devices and for different purposes. Therefore, it is important to study as many ML models as possible, evaluating their effectiveness and efficiency regarding various real-world datasets so that users can identify the most appropriate models based on their unique business nature and security requirements. Due to the unavailability of appropriate datasets and the scope of our current project, we did not consider other machine learning models, such as Elastic Regression, Lasso Regression, Ridge Regression, Random Forest, and Naive Bayes which have been studied in some recent publications [7].

7. Conclusions and Future Work

The vulnerable and resource-constrained end devices that are used in emerging IoT applications make the security of these applications a great challenge. Anomaly detection is a crucial security procedure that protects IoT applications through the identification of unusual or abnormal behavior. In this paper, we investigated anomaly detection powered by machine learning algorithms. In view of the lack of studies identifying optimal anomaly detection models for the IoT, our work investigates the effectiveness and efficiency of various models. Three representative machine learning models were studied in this paper, namely XGBoost, SVMs and DCNNs. Evaluating ML-powered anomaly detection models using distinct datasets collected from different environments is vital. Therefore, multiple datasets containing data from diverse environments were utilized. These datasets included the IoT-23 dataset, NSL-KDD dataset, and TON_IoT dataset.

The experimental results proved that XGBoost is the most effective method, achieving accuracies of up to 99.98%. The DCNN performed second best, achieving the highest accuracy of 99.90%. The SVM was the third best algorithm, achieving accuracies of up to 96.71%. Although the SVM and DCNN both achieved good accuracies, XGBoost performed well above these two models in our evaluation. In particular, the highest accuracies were found when executing these methods on the IoT-23 dataset. XGBoost also proved to be the most efficient regarding the training time. XGBoost took an average of 0.7748 s for training and only 0.0032 s for testing. The SVM took an average of 556.11 s for training and 79.06 s for testing. As for the average across all DCNN architectures, the training time was about 9916.9 s and the testing time was 342.49 s.

These models have further been assessed using our real-world IoT data collected from an IoT testbed consisting of various physical devices. Our evaluation of the anomaly detection models using real-world data proves that XGBoost can efficiently and accurately detect anomalies in real-world IoT applications. However, with the many ML models and unique IoT environments, it is difficult to conclude which model works most effectively in all IoT systems with different devices. We may extend our research to include more

well-known ML models in our future projects. We believe more research findings will be achieved which will be beneficial for industrial and organizational IoT users.

Author Contributions: Conceptualization, M.B., W.F., X.-W.W., S.E. and Z.G.; methodology, M.B. and Z.G.; software, M.B. and Z.G.; validation, M.B. and Z.G.; formal analysis, M.B. and Z.G.; investigation, M.B. and Z.G.; resources, W.F., X.-W.W. and S.E.; data curation, M.B.; writing—original draft preparation, M.B.; writing—review and editing, M.B.; visualization, M.B.; supervision, W.F., X.-W.W. and S.E.; project administration, W.F., X.-W.W. and S.E.; funding acquisition, W.F. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the NSA-NCAE-C grants H98230-20-1-0296 and H98230-22-1-0315.

Data Availability Statement: Publicly available datasets were analyzed in this study. These data can be found here: <https://www.stratosphereips.org/datasets-iot23> (accessed on 18 February 2021) [6], <https://www.unb.ca/cic/datasets/nsl.html> (accessed on 18 February 2021) [7], and <https://research.unsw.edu.au/projects/toniot-datasets> (accessed on 18 February 2021) [8].

Acknowledgments: The authors extend their thanks to Larry Pearlstein, TCNJ, for the use of Zelda. In addition, the authors would like to thank members of the IUP IoT Research Team.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Hossain, M.; Kayas, G.; Hasan, R.; Skjellum, A.; Noor, S.; Islam, S.M.R. A Holistic Analysis of Internet of Things (IoT) Security: Principles, Practices, and New Perspectives. *Future Internet* **2024**, *16*, 40. [CrossRef]
- Cole, T. Interview with Kevin Ashton—Inventor of IoT: Is Driven by the Users. Available online: <https://www.avnet.com/wps/portal/silica/resources/article/interview-with-iot-inventor-kevin-ashton-iot-is-driven-by-the-users/> (accessed on 1 April 2022).
- Al-Hejri, I.; Azzedin, F.; Almuhammadi, S.; Eltoweissy, M. Lightweight Secure and Scalable Scheme for Data Transmission in the Internet of Things. *Arab. J. Sci. Eng.* **2024**. [CrossRef]
- Vailshery, L.S. Global IoT and Non-IoT Connections 2010–2025. Available online: <https://www.statista.com/statistics/1101442/iot-number-of-connected-devices-worldwide/> (accessed on 1 April 2022).
- Posey, B.; Shea, S. What Are IoT Devices?—Definition from Techtarget.com. Available online: <https://internetofthingsagenda.techtarget.com/definition/IoT-device> (accessed on 1 April 2022).
- Shea, S.; Wigmore, I. IoT Security (Internet of Things Security). Available online: <https://www.techtarget.com/iotagenda/definition/IoT-security-Internet-of-Things-security> (accessed on 1 April 2022).
- Wu, X.W.; Cao, Y.; Dankwa, R. Accuracy vs Efficiency: Machine Learning Enabled Anomaly Detection on the Internet of Things. In Proceedings of the IEEE International Conference on Internet of Things and Intelligence Systems, Bali, Indonesia, 24–26 November 2022; pp. 245–251.
- Fraihat, S.; Makhadmeh, S.; Awad, M.; Al-Betar, M.A.; Al-Redhaei, A. Intrusion detection system for large-scale IoT NetFlow networks using machine learning with modified Arithmetic Optimization Algorithm. *Internet Things* **2023**, *22*, 100819. [CrossRef]
- Awad, M.; Fraihat, S.; Salameh, K.; Al Redhaei, A. Examining the Suitability of NetFlow Features in Detecting IoT Network Intrusions. *Sensors* **2022**, *22*, 6164. [CrossRef] [PubMed]
- Garcia, S.; Parmisano, A.; Erquiaga, M.J. IoT-23: A Labeled Dataset with Malicious and Benign IoT Network Traffic (Version 1.0.0). 2020. Available online: <https://www.stratosphereips.org/datasets-iot23> (accessed on 18 February 2021).
- NSL-KDD Dataset. Available online: <https://www.unb.ca/cic/datasets/nsl.html> (accessed on 18 February 2021).
- TON_IoT Datasets. Available online: <https://research.unsw.edu.au/projects/toniot-datasets> (accessed on 18 February 2021).
- Hossain, M.T.; Imran, M.A. ToN-IoT: A dataset for traffic analysis of IoT devices. In Proceedings of the IEEE International Conference on Communications, Kansas City, MO, USA, 20–24 May 2018; pp. 1–6.
- Alsaedi, A.; Moustafa, N.; Tari, Z.; Mahmood, A.; Anwar, A. TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems. *IEEE Access* **2022**, *8*, 165130–165150. [CrossRef]
- Cañedo, J.; Skjellum, A. Using Machine Learning to secure IoT systems. In Proceedings of the 2016 14th Annual Conference on Privacy, Security and Trust (PST), Auckland, New Zealand, 12–14 December 2016; pp. 219–222.
- Hussain, F.; Hussain, R.; Hassan, S.A.; Hossain, E. Machine Learning in IoT security: Current solutions and future challenges. *IEEE Commun. Surv. Tutor.* **2020**, *22*, 1686–1721. [CrossRef]
- Dalal, K.R. Analyzing the role of supervised and unsupervised Machine Learning in IoT. In Proceedings of the 2020 International Conference on Electronics and Sustainable Communication Systems (ICESC), Coimbatore, India, 2–4 July 2020; pp. 75–79.
- Vitorino, J.; Andrade, R.; Praca, I.; Sousa, O.; Maia, E. A Comparative Analysis of Machine Learning Techniques for IoT Intrusion Detection. In Proceedings of the 14th International Symposium on Foundations and Practice of Security (FPS 2021), Paris, France, 7–10 December 2021; pp. 191–207.
- Diro, A.; Chilamkurthi, N.; Nguyen, V.D.; Heyne, W. A Comprehensive Study of Anomaly Detection Schemes in IoT Networks Using Machine Learning Algorithms. *Sensors* **2021**, *21*, 8320. [CrossRef] [PubMed]

20. Balega, M.; Farag, W.; Ezekiel, S.; Wu, X.-W.; Deak, A.; Good, Z. IoT Anomaly Detection Using a Multitude of Machine Learning Algorithms. In Proceedings of the 2022 IEEE Applied Imagery Pattern Recognition Workshop, Washington, DC, USA, 11–13 October 2022; pp. 1–6.
21. Good, Z.; Farag, W.; Wu, X.-W.; Ezekiel, S.; Balega, M.; May, F.; Deak, A. Comparative Analysis of Machine Learning Techniques for IoT Anomaly Detection Using the NSL-KDD Dataset. *Int. J. Comput. Sci. Netw. Secur.* **2023**, *23*, 46–52.
22. What Is Machine Learning? Available online: <https://www.ibm.com/cloud/learn/machine-learning> (accessed on 1 April 2022).
23. Chen, T.; Guestrin, C. XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; pp. 785–794.
24. Hastie, T.; Tibshirani, R.; Friedman, J.H.; Friedman, J.H. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*; Springer: New York, NY, USA, 2009; Volume 2.
25. Chang, W.; Liu, Y.; Xiao, Y.; Yuan, X.; Xu, X.; Zhang, S.; Zhou, S. A Machine Learning based prediction method for hypertension outcomes based on medical data. *Diagnostics* **2019**, *9*, 178. [[CrossRef](#)] [[PubMed](#)]
26. XGBoost Documentation. Available online: <https://xgboost.readthedocs.io/en/stable/index.html> (accessed on 1 June 2021).
27. Vapnik, V. *Estimation of Dependences Based on Empirical Data*; Springer: New York, NY, USA, 2006. [[CrossRef](#)]
28. Jakkula, V. *Tutorial on Support Vector Machine (SVM)*; School of EECS, Washington State University: Pullman, WA, USA, 2006; Volume 37, p. 3.
29. Pupale, R. Support Vector Machines (SVM)—An Overview. Available online: <https://towardsdatascience.com/https-medium-com-pupalerushikesh-svm-f4b42800e989> (accessed on 1 April 2022).
30. Deep Convolutional Neural Networks. Available online: <https://www.run.ai/guides/deep-learning-for-computer-vision/deep-convolutional-neural-networks> (accessed on 1 April 2022).
31. Stoian, N. Machine Learning for Anomaly Detection in IoT Networks: Malware Analysis on the IoT-23 Dataset. Bachelor’s Thesis, University of Twente, Enschede, The Netherlands, 2020.
32. Lippmann, R.; Fried, D.; Graf, I.; Haines, J.; Kendall, K.; McClung, D.; Weber, D.; Webster, S.; Wyschogrod, D.; Cunningham, R.; et al. Evaluating intrusion detection systems: The 1998 darpa offline intrusion detection evaluation. In Proceedings of the DARPA Information Survivability Conference and Exposition, DISCEX’00, Hilton Head, SC, USA, 25–27 January 2000; Volume 2, pp. 12–26.
33. Tavallaee, M.; Bagheri, E.; Lu, W.; Ghorbani, A.A. A detailed analysis of the KDD Cup 99 dataset. In Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications, Ottawa, ON, Canada, 8–10 July 2009; pp. 1–6.
34. Revathi, S.; Malathi, A. A detailed analysis on the NSL-KDD dataset using various machine learning techniques for intrusion detection. *Int. J. Eng. Res. Technol.* **2013**, *2*, 1848–1853.
35. Moustafa, N.; Keshky, M.; Debiez, E.; Janicke, H. Federated TON_IoT Windows Datasets for Evaluating AI-Based Security Applications. In Proceedings of the IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), Guangzhou, China, 29 December 2020; pp. 848–855.
36. Hale, J. The 3 Most Important Composite Classification Metrics. Available online: <https://towardsdatascience.com/the-3-most-important-composite-classification-metrics-b1f2d886dc7b> (accessed on 1 April 2022).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.